

TECHNICAL DOCUMENTATION: AI TREND HUB

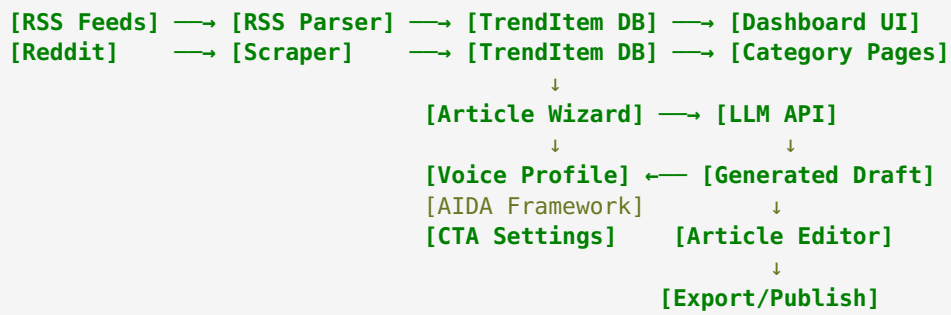
Generated: April 13, 2026

1. System Architecture

1.1 Stack Overview

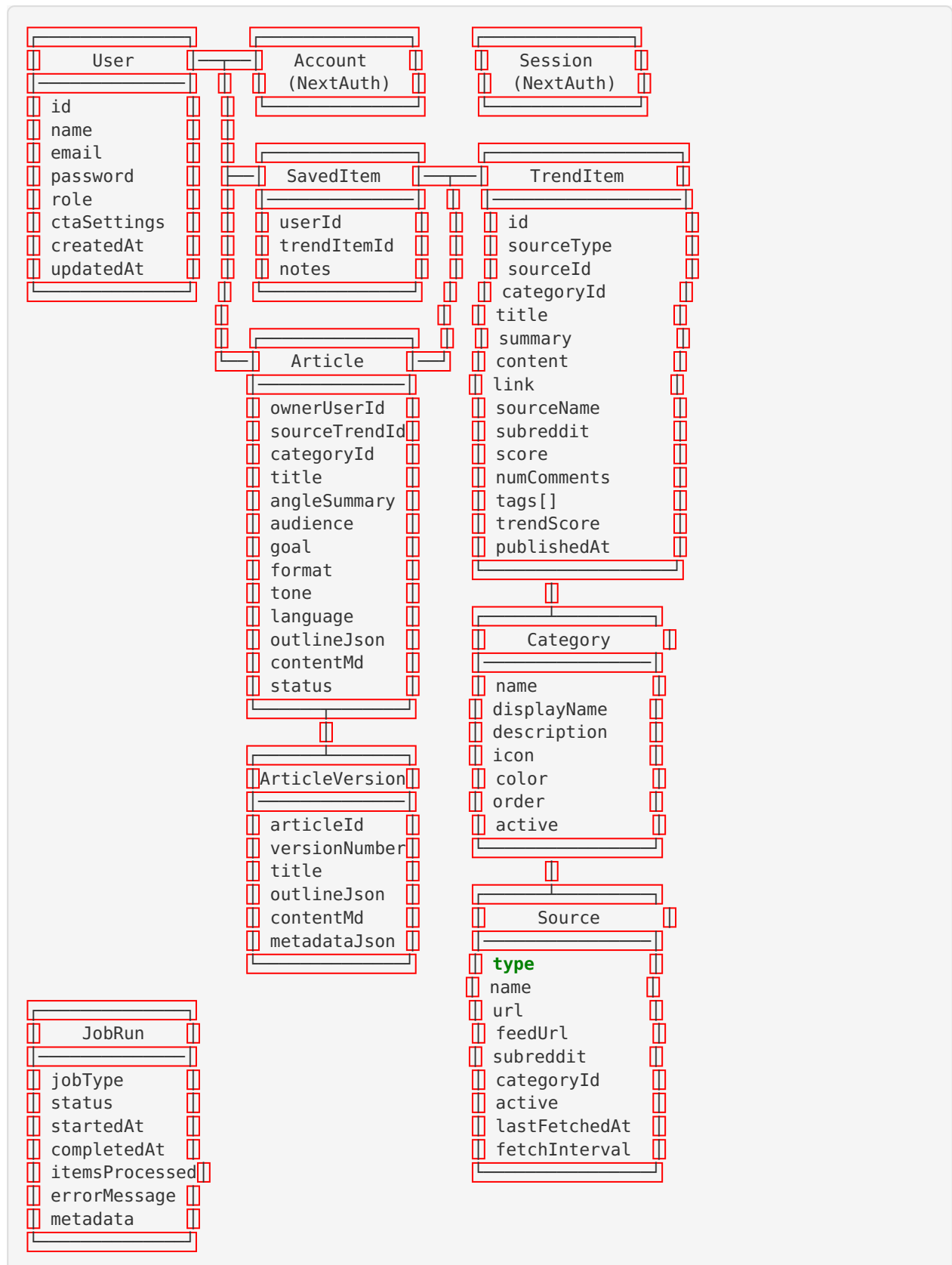
Layer	Technology	Version/Details
Frontend	Next.js (App Router)	14.x with TypeScript
UI Framework	TailwindCSS	Dark theme, card-based layout
UI Components	shadcn/ui	80+ pre-built components
Charts	Recharts	Time series and category visualizations
Backend	Next.js API Routes	RESTful endpoints
Database	PostgreSQL	Hosted by Abacus AI
ORM	Prisma	Type-safe database access
Authentication	NextAuth.js	JWT-based, credentials provider
AI/LLM	Abacus.AI API	gpt-4.1-mini model
Package Manager	Yarn	For dependency management

1.2 Data Flow



2. Database Schema

2.1 Entity Relationship Diagram



2.2 Key Indexes

- `TrendItem` : `categoryId`, `sourceType`, `publishedAt`, `trendScore`
 - `SavedItem` : `userId` (+ unique constraint on `userId+trendItemId`)
 - `Article` : `ownerUserId`, `sourceTrendItemId`, `status`, `createdAt`
 - `ArticleVersion` : `articleId` (+ unique constraint on `articleId+versionNumber`)
-

3. API Reference

3.1 Authentication

POST /api/auth/[...nextauth]

- NextAuth.js handler
- Supports: `signIn`, `signOut`, `session`, `csrf`
- Provider: `credentials` (email + password)
- Session strategy: JWT

POST /api/signup

- Body: `{ email, password, name? }`
- Returns: `{ user: { id, email, name, role } }`
- Hashes password with bcrypt

3.2 Trends

GET /api/trends

- Query params: `category`, `timeRange` (24h/7d/30d), `search`, `page`, `limit`
- Returns: `{ items: TrendItem[], total: number, page: number }`

3.3 Categories

GET /api/categories

- Returns: `{ categories: Category[] }`

3.4 Saved Items

GET /api/saved

- Returns user's saved items with trend item details

POST /api/saved

- Body: `{ trendItemId: string }`
- Toggles save/unsave

3.5 Dashboard

GET /api/dashboard/stats

- Returns: { totalItems, newToday, savedCount, categories: [...], recentItems: [...], chartData: [...] }

3.6 Articles

GET /api/articles

- Query: status, category, search, page, limit
- Returns: { articles: Article[], total: number }

POST /api/articles

- Body: { title, sourceTrendItemId?, categoryId?, ... }
- Creates new article record

GET /api/articles/[id]

- Returns full article with versions

PUT /api/articles/[id]

- Updates article content, status, metadata

DELETE /api/articles/[id]

- Soft deletes or permanently removes article

POST /api/articles/angles

- Body: { trendItemId, audience, goal, tone, format, language }
- Returns: { angles: [{ angle: string, title: string }] }
- Uses LLM API to generate 3-5 angle suggestions

POST /api/articles/outline

- Body: { trendItemId, angle, title, audience, goal, tone, format, language }
- Returns: { outline: { introduction, sections[], conclusion } }

POST /api/articles/draft

- Body: { trendItemId, angle, title, outline, audience, goal, tone, format, language, useAida }
- Returns: { content: string (markdown), articleId, versionId }
- Integrates: Voice Profile, AIDA Framework, CTA Settings, Cross-Industry IP

POST /api/articles/section-regenerate

- Body: { articleId, sectionIndex, instructions }
- Returns: { section: string }

POST /api/articles/[id]/social-posts

- **STUB** - Returns placeholder social posts

3.7 Settings

GET /api/settings/cta

- Returns: `{ ctaSettings: CTASettings }`

POST /api/settings/cta

- Body: `{ ctaSettings: CTASettings }`
- Saves CTA configuration to user profile

CTASettings Structure:

```
{
  enabled: boolean,
  book: { enabled, title, description, buyLink, price },
  video: { enabled, title, description, videoLink },
  affiliateProducts: [{ id, name, description, link, commission }],
  customCTAs: [{ id, text, link, description }],
  placement: 'end' | 'middle' | 'both',
  style: 'invitational' | 'direct' | 'educational'
}
```

4. Voice Profile System

4.1 Architecture

The voice profile system (`lib/voice-profile.ts`) is a comprehensive configuration object that feeds into the LLM prompt during article generation.

Components:

1. **Core Philosophy** - Jack's beliefs and positioning
2. **Intellectual Property** - 7 named frameworks with descriptions
3. **Writing Style DNA** - Tone, structure, vocabulary, patterns
4. **Cross-Industry Applications** - Framework translations for 8+ industries
5. **Case Studies** - Real examples from Jack's experience

4.2 Industry Detection

The draft route (`app/api/articles/draft/route.ts`) uses `detectIndustryContext()` to automatically identify which industry an article targets based on keywords in the title and angle. It then injects the relevant framework translations from the cross-industry applications map.

Supported Industries:

- `recruitment` (origin)
- `sales_b2b`
- `ecommerce_retail`
- `restaurants_hospitality`
- `healthcare`
- `manufacturing_logistics`

- `professional_services`
- `saas`

4.3 AIDA Integration

When the `useAida` flag is true:

1. AIDA instructions are injected into the LLM prompt
 2. Guardrail instructions prevent salesy tone
 3. Tone calibration adjusts AIDA intensity by article goal
 4. A 5-question authenticity check is included
-

5. Background Job System

5.1 Job Scheduler

File: `lib/job-scheduler.ts`

- Runs every 30 minutes
- Prevents concurrent execution
- Tracks status in `JobRun` table
- Supports manual trigger via `/api/jobs/refresh`

5.2 RSS Parser

File: `lib/rss-parser.ts`

- Fetches and normalizes RSS feeds
- Extracts tags and calculates trend scores
- Deduplicates items via database checks
- Sources: TechCrunch AI, MIT Tech Review, VentureBeat AI, etc.

5.3 Reddit Scraper

File: `lib/reddit-scraper.ts`

- Web scraping implementation (no API key needed)
 - Filters AI-related posts via keyword matching
 - Calculates Reddit-specific scoring (upvotes + comments)
 - Subreddits: `r/MachineLearning`, `r/LocalLLaMA`, `r/ChatGPT`, etc.
-

6. Frontend Components

6.1 Dashboard Components

Component	File	Purpose
Header	<code>components/dashboard/header.tsx</code>	Top navigation bar
Sidebar	<code>components/dashboard/sidebar.tsx</code>	Side navigation
StatsCard	<code>components/dashboard/stats-card.tsx</code>	Numeric highlight tiles
Filters	<code>components/dashboard/filters.tsx</code>	Search + filter controls
TrendCard	<code>components/dashboard/trend-card.tsx</code>	Individual trend item
TimeSeriesChart	<code>components/dashboard/time-series-chart.tsx</code>	Time-based visualizations
CategoryChart	<code>components/dashboard/category-chart.tsx</code>	Category breakdowns

6.2 Article Components

Component	File	Purpose
ArticleWizard	<code>components/articles/article-wizard.tsx</code>	4-step creation wizard
ArticleEditor	<code>components/articles/article-editor.tsx</code>	Full editor with versioning

6.3 Wizard Steps

1. **Basics** - Audience, goal, format, tone, language, AIDA toggle
 2. **Angles** - AI generates 3-5 angles with titles; user selects one
 3. **Outline** - AI generates structured outline; user can edit, reorder, add/delete sections
 4. **Draft** - AI generates full article; opens in editor
-